

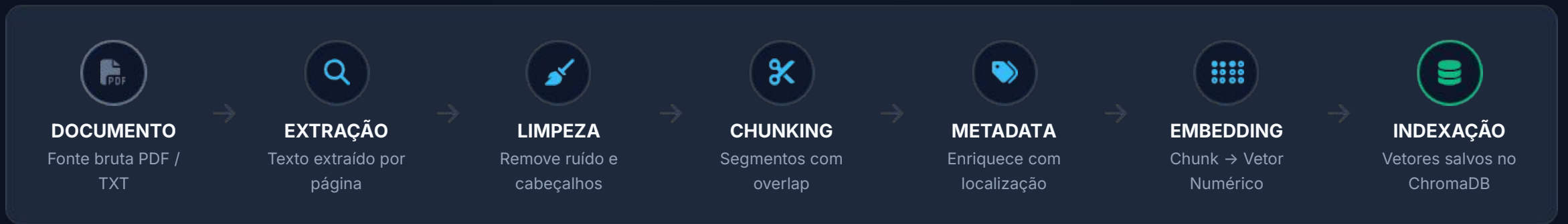
O Pipeline de Dados e Busca

Da bagunça documental ao contexto semântico preciso



Cobre: Ingestão & Indexação + Recuperação Semântica

Fase Offline: o ETL dos seus documentos



Nunca misturar com a fase online Roda offline, uma vez (ou por atualização)

Referências: `src/1_ingestion.py` · `src/2_indexing.py`

PDFs institucionais: o inimigo dos embeddings

O que parece no PDF (Visual):

Universidade Federal — Pró-Reitoria de Graduação
Resolução nº 023/2025 — Página 14 de 47

Art. 14. O prazo máximo para solicitação de revisão de nota é de 10 (dez) dias úteis após a divulgação dos resultados...

Universidade Federal — Pró-Reitoria de Graduação

Resolução nº 023/2025 — Página 14 de 47

O que o extrator vê (Texto Bruto):

Texto bruto extraído sem limpeza prévia:

"Universidade Federal – Pró-Reitoria de Graduação
Resolução nº 023/2025 – Página 14 de 47

Art. 14. O prazo máximo para solicitação de revisão de nota é de 10 (dez) dias úteis após a divulgação dos resultados...

Universidade Federal – Pró-Reitoria de Graduação
Resolução nº 023/2025 – Página 14 de 47"

⚠ **Cabeçalhos e rodapés repetidos envenenam o embedding.** O chunk fica com 40% de ruído institucional e 60% de conteúdo real. O embedding captura ambos, e a busca semântica falha.

Limpeza: texto sujo gera embedding sujo

```
39 # -----
40 # Limpeza de Texto
41 # -----
42
43 def clean_text(raw_text: str) -> str:
44     """
45     Normaliza e limpa texto bruto
46     """
47     # 1. Normalização Unicode
48     text = unicodedata.normalize("NFKC", raw_text)
49
50     # 2. Remove soft-hyphens (\u00ad) - invisíveis mas tóxicos
51     text = text.replace("\u00ad", "")
52
53     # 3. Junta palavras quebradas por hífen no fim da linha
54     text = re.sub(r"(\w)-\s*\n\s*(\w)", r"\1\2", text)
55
56     # 4. Colapsa espaços e tabs excessivos
57     text = re.sub(r"[ \t]+", " ", text)
58
59     # 5. Normaliza quebras de linha
60     text = re.sub(r"\n{3,}", "\n\n", text)
61
62     return text.strip()
```

- A função `clean\_text` é responsável por "higienizar" o texto extraído, eliminando ruídos que prejudicariam a qualidade da busca semântica e dos embeddings.
- Ela realiza etapas críticas como a normalização Unicode (garantindo que caracteres idênticos visualmente tenham a mesma representação de bytes) e a remoção de *soft-hyphens* (hifens invisíveis comuns em PDFs).
- Ao colapsar múltiplos espaços e juntar palavras que foram quebradas no fim de uma linha, a função preserva a integridade semântica do texto e economiza tokens preciosos no contexto do LLM.

O \u00ad (soft-hyphen) não aparece na tela do usuário, mas aparece no embedding. É invisível para humanos e tóxico para a IA.

Chunking: sliding window sobre o texto

src/1_ingestion.py

```
121 # -----
122 # Chunking com Sobreposição (Sliding Window)
123 # -----
124
125 def split_into_chunks(
126     text: str,
127     source: str,
128     page: int,
129     doc_chunk_offset: int = 0,
130     cfg: ChunkingConfig = chunking_cfg,
131 ) -> list[Chunk]:
132     """
133     Divide um texto em chunks com sobreposição usando janela deslizante.
134     """
135     chunks: list[Chunk] = []
136     step = cfg.chunk_size - cfg.chunk_overlap
137     source_name = Path(source).name
138
139     start = 0
140     chunk_idx = doc_chunk_offset
141
142     while start < len(text):
143         end = start + cfg.chunk_size
144         chunk_text = text[start:end].strip()
145
146         if len(chunk_text) >= cfg.min_chunk_size:
147             chunk_id = f"{source_name}_{chunk_idx}"
148             chunks.append(
149                 Chunk(
150                     id=chunk_id,
151                     text=chunk_text,
152                     source=source,
153                     page=page,
154                     chunk_idx=chunk_idx,
155                 )
156             )
157             chunk_idx += 1
158
159         start += step
160
161     return chunks
```

Texto completo da página normalizada:

Art. 13... Art. 14... Art. 15...

↓ chunk_with_overlap()

Art. 13... Art. 14. 0 → 1200

Art. 14. 0 prazo... 1000 → 2200

Art. 15... 2000 → 3200

↑ O **overlap** (200c) garante que o contexto não sofra cortes secos

Tamanho do chunk: a decisão de maior impacto

Não existe tamanho universalmente correto. Existe o tamanho certo para o seu tipo de documento e caso de uso específico.

- **Fixa (Tamanho Rígido):** Divide o texto estritamente por uma quantidade exata de caracteres. É a opção mais rápida de processar, mas pode cortar palavras ou frases importantes pelo meio.
- **Estrutural (Parágrafos e Pontos):** Divide o texto respeitando os limites naturais de parágrafos e pontuações. Evita quebrar o sentido das frases e é o método mais equilibrado para a maioria dos projetos.
- **Semântica (Por Contexto):** Analisa o significado do texto e só realiza o corte quando detecta que o assunto mudou. Garante a máxima precisão do contexto original, embora exija mais processamento.

☰ Heurísticas de partida:

- **Textos corridos:** 1000–2000 chars (overlap 10-20%)
- **Docs estruturados:** Chunking semântico por seção/artigo

⚠ **Cuidado:** Um chunk de 5000 chars não é "mais rico". É um vetor que mistura assuntos. O LLM precisa de precisão, não volume. Prefira chunks focados e um Top-K maior.

Metadados: governança embutida em cada chunk

💬 Pense como um *covering index* relacional. O embedding é a chave de busca. A metadata é o payload retornado junto — sem custo extra de JOIN, pois já reside no Vector DB.

Objeto Chunk Estruturado

```
chunks.append(  
    Chunk(  
        id=chunk_id,  
        text=chunk_text,  
        source=source,  
        page=page,  
        chunk_idx=chunk_idx,  
    )  
)
```

Estrutura e Metadados de um Chunk:

- **id:** Identificador único gerado para controlar e rastrear individualmente cada bloco de texto dentro do banco de dados vetorial.
- **text:** O trecho de texto fatiado propriamente dito, que será convertido em vetor (embedding) e enviado como contexto para o LLM.
- **source:** A fonte ou nome do arquivo original, garantindo a rastreabilidade e a transparência de onde a informação veio.
- **page:** O número da página de onde o trecho foi extraído, permitindo que o modelo de IA cite a página exata ao responder ao usuário.
- **chunk_idx:** O índice sequencial do bloco dentro do documento, essencial para ordenar os trechos e recuperar o contexto vizinho (o que vem antes ou depois) se necessário.

Sem metadata, o RAG é uma caixa preta inauditaável

Metadata não entra no vetor numérico — mas é o que torna o RAG institucional viável em produção.



Busca Cirúrgica (Filtrar)

Buscar apenas em documentos vigentes de 2025, do tipo "resolução".

```
where={"year": 2025, "type": "resolucao"}
```



Rastreabilidade (Citar)

A resposta gerada cita sua origem: *"Fonte: Regimento 2025 / pág. 14"*. O usuário pode clicar e validar contra a fonte oficial original.



Trilha de Acesso (Auditar)

Os logs do sistema registram qual chunk foi usado para embasar cada resposta enviada. Exigência fundamental de Compliance e LGPD.



Reindexação Segura (Versionar)

pipeline_version + doc_hash garantem a identificação exata de qual documento gerou aquele vetor, evitando bancos de dados "sujos".

Indexação: persistência atômica no ChromaDB

Com os chunks segmentados e enriquecidos com metadados, o passo final da Fase Offline é gravá-los de forma persistente no banco de dados vetorial.

src/2_indexing.py

```
97     for i in range(0, len(chunks), batch_size):
98         batch = chunks[i : i + batch_size]
99         batch_texts = [c.text for c in batch]
100
101         # Gera embeddings via Ollama
102         batch_embeddings = get_embeddings(batch_texts)
103
104         collection.upsert(
105             ids=[c.id for c in batch],
106             embeddings=batch_embeddings,
107             documents=batch_texts,
108             metadatas=[
109                 {
110                     "source": c.source,
111                     "page": c.page,
112                     "chunk_idx": c.chunk_idx,
113                     "doc_hash": c.doc_hash,
114                     "status": "vigente",
115                     "version": "v1.0"
116                 }
117                 for c in batch
118             ],
119         )
120         progress.advance(task, len(batch))
```

```
14 def get_embeddings(texts: list[str]) -> list[list[float]]:
15     """
16     Gera embeddings para uma lista de textos usando o Ollama.
17     """
18     embeddings = []
19     for text in texts:
20         try:
21             response = ollama.embeddings(model=ollama_cfg.embed_model, prompt=text)
22             embeddings.append(response["embedding"])
23         except Exception as e:
24             console.print(f"[red]Erro ao gerar embedding via Ollama: {e}[/red]")
25             raise
26     return embeddings
```

Na prática: o pipeline offline rodando

```
Terminal - bash
```

 **Ingestão de Documentos**

Documentos Encontrados

Arquivo	Tipo	Tamanho
codigo_etica_conduta.md	.MD	3.0 KB
regimento_interno.txt	.TXT	4.1 KB
politica_seguranca_informacao.md	.MD	3.3 KB

```
? Indexar 3 arquivo(s) no ChromaDB? Yes
```

 **Processando:** codigo_etica_conduta.md

✓ 3 chunks totais até agora

 **Processando:** regimento_interno.txt

✓ 7 chunks totais até agora

 **Processando:** politica_seguranca_informacao.md

✓ 11 chunks totais até agora

 **Gerando embeddings e inserindo no ChromaDB (11 chunks)...**

Indexando... 100%

✓ **Indexação concluída! Total na coleção: 11**

 **Resumo da Ingestão**

✓ **Ingestão concluída com sucesso!**

 Arquivos processados: 3

 Chunks gerados: 11

 Total na coleção: 11

3 arquivos → 11 chunks

Com janela de 1200 chars e overlap de 200, geramos uma média de ~2,8 chunks matemáticos por arquivo original.

localhost:11434

Utilizando o Ollama rodando localmente. Zero tráfego de rede externo ou chamadas de API cloud pagas durante toda a ingestão institucional.

FASE ONLINE

Recuperação

Do banco vetorial ao contexto entregue ao LLM

[Offline:  Concluído]

Doc → Extração → Chunk → Index

[Online: → Agora]

Pergunta → Busca → Contexto → LLM

↑ Estamos aqui

"O retrieval é mais determinístico que o modelo.
Conserte o retrieval primeiro."

Por que retrieval importa mais do que o modelo

O LLM é estocástico. O retrieval é determinístico — dada a mesma pergunta e índice, **sempre retorna os mesmos chunks**. Erros de retrieval são rastreáveis e resolvíveis.

Erro de Retrieval

Chunk *errado* recuperado → LLM responde com base em evidência errada.

- > **Causa:** Chunking ruim, embedding fraco, filtros ausentes.
- > **Diagnóstico:** Inspeção os chunks do top-K diretamente (ignore a IA).
- > **Correção:** Ajuste o pipeline offline → reindexe.

Erro de LLM / Prompt

Chunk *correto* recuperado → LLM ignora a evidência e alucina.

- > **Causa:** Prompt fraco sem instrução de grounding rígida.
- > **Diagnóstico:** Compare a resposta com o texto exato do chunk.
- > **Correção:** Ajuste o prompt na requisição online da API.

Dúvida de onde está o erro? Inspeção o top-K retornado primeiro. Sempre.

Top-K: a query do banco vetorial

```
</> SELECT * FROM chunks ORDER BY cosine_similarity(chunk, query) DESC LIMIT k
```

Fazer Pergunta ao RAG

11 chunks disponíveis | Modelo: llama3.2

? Sua pergunta (ou 'sair' para voltar ao menu): quero tirar ferias, quando e como posso?

🔗 5 fonte(s) recuperada(s)

#	Score	Fonte	Pág.	Prévia
1	0.67	regimento_interno.txt	1	# Resolução Fictícia no 001/2025 ## Conselho Administrativo – Regras d...
2	0.61	regimento_interno.txt	1	nda de 1/3) O funcionário poderá, a seu critério, vender até 1/3 (um ...
3	0.59	politica_seguranca_informacao.md	1	adores que caírem em mais de 2 (duas) simulações no ano deverão obriga...
4	0.58	codigo_etica_conduta.md	1	# Código de Ética e Conduta Corporativa ## Código Geral de Diretrizes ...
5	0.58	politica_seguranca_informacao.md	1	# Política de Segurança da Informação (PSI) – Diretrizes Corporativas ...

★QUERY: "prazo para pedir revisão"

K = 1 "Art. 14 — revisão de nota"

K = 2 "Art. 15 — prazo recurso"

K = 3 "Art. 16 — aprovação"

K = 4 "resolução 001 capa" ⚠ Corte

K = 5 "ata reunião geral" ❌ Corte

No código: como a busca semântica é executada

src/3_retrieval.py

```
56     """
57     Busca os chunks mais relevantes com suporte a filtros e re-ranking.
58     """
59     # 1. Vetorizar a query via Ollama
60     query_embedding = get_single_embedding(query)
61
62     # 2. Buscar no ChromaDB (pede mais se for re-ranquear para ter margem)
63     k_initial = cfg.top_k * 5 if cfg.rerank else cfg.top_k
64     n_results = min(k_initial, collection.count())
65
66     if n_results == 0:
67         return []
68
69     results = collection.query(
70         query_embeddings=[query_embedding],
71         n_results=n_results,
72         where=where,
73         include=["documents", "metadatas", "distances"],
74     )
```

Mecanismo de Busca Vetorial e Re-ranking

- **Vetorização da Query:** Transforma a pergunta textual do usuário em coordenadas matemáticas (embedding) para permitir a comparação semântica com os documentos do banco de dados.
- **Busca e Filtragem Híbrida:** Consulta o banco aplicando filtros específicos de metadados e recupera uma quantidade ampliada de blocos para gerar uma margem de segurança antes do refinamento.

O que você agora sabe (Checklist do Bloco)

🔹 Ingestão (Offline)

- Extrair PDF por página preserva a localização.
- Texto sujo destrói o vetor. Normalizar é regra.
- Chunking com tamanho errado destrói o recall.
- Metadata não entra no vetor, mas torna o RAG auditável e apto para produção.

🔍 Recuperação (Online)

- Top-K é determinístico. Inspecione a query.
- Filtro de metadata atua como o WHERE clause.
- Top-K alto → mais Recall.
Re-ranking → mais Precision.
- A similaridade do cosseno compara ângulos semânticos.



Próximo passo — Bloco 3: Integração e Geração O contexto foi recuperado com precisão. Agora: como transformar esses chunks em uma resposta confiável — o prompt RAG, janela de contexto, integração de API e avaliação prática do sistema.